

Assignment 3: Data Exploration

Juanying Huangfu

Spring 2026

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on Data Exploration.

Directions

1. Rename this file `<FirstLast>_A03_DataExploration.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. [NEW] Assign a useful **name to each code chunk** and include ample **comments** with your code.
5. Be sure to **answer the questions** in this assignment document.
6. When you have completed the assignment, **Knit** the text and code into a single PDF file.
7. After Knitting, submit the completed exercise (PDF file) to Canvas.
8. Initial here to acknowledge that you did not use AI in completing this assignment, except where expressly allowed: __Juanying Huangfu__

TIP: If your code extends past the page when knit, tidy your code by manually inserting line breaks in your code chunks.

TIP: If your code fails to knit, check: * That no `install.packages()` or `View()` commands exist in your code. * That you are not displaying the entire contents of a large dataframe in your code.

Set up your R session

1. Load necessary packages (tidyverse, here), check your current working directory and import two datasets: the ECOTOX neonicotinoid dataset (`ECOTOX_Neonicotinoids_Insects_raw.csv`) and the Niwot Ridge NEON dataset for litter and woody debris (`NEON_NIWO_Litter_massdata_2018-08_raw.csv`). Name these datasets “Neonics” and “Litter”, respectively.

Be sure to: * Use the `here()` package in specifying the paths to your datasets * Include the appropriate subcommand to read in character based columns as factors

```
# Load necessary packages
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.3      v tidyr     1.3.1
## v purrr      1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(here)
```

```
## here() starts at /home/guest/EDE_Spring2026
```

```
# Check current working directory
here()
```

```
## [1] "/home/guest/EDE_Spring2026"
```

```
# Import datasets
Neonics <- read.csv(
  here("Data", "Raw", "ECOTOX_Neonicotinoids_Insects_raw.csv"),
  stringsAsFactors = TRUE
)

Litter <- read.csv(
  here("Data", "Raw", "NEON_NIWO_Litter_massdata_2018-08_raw.csv"),
  stringsAsFactors = TRUE
)
```

Learn about your system

2. QUESTION: The neonicotinoid dataset was collected from the Environmental Protection Agency's ECOTOX Knowledgebase, a database for ecotoxicology research. Neonicotinoids are a class of insecticides used widely in agriculture. The dataset that has been pulled includes all studies published on insects. Why might we be interested in the ecotoxicology of neonicotinoids on insects? Feel free to do a brief internet search if you feel you need more background information. (AI is allowed here, but put answers in your own words.)

Answer: Neonicotinoids are meant to control insect pests, but they don't only affect the insects they're targeting. Many studies have shown that these chemicals can also harm non-target insects, including pollinators and other beneficial species. Since insects play key roles in pollination, food webs, and ecosystem functioning, even small toxic effects can scale up to broader ecological and agricultural impacts. That's why understanding how neonicotinoids affect insects is important beyond just pest control.

3. QUESTION: The Niwot Ridge litter and woody debris dataset was collected from the National Ecological Observatory Network, which collectively includes 81 aquatic and terrestrial sites across 20 ecoclimatic domains. 32 of these sites sample forest litter and woody debris, and we will focus on the Niwot Ridge long-term ecological research (LTER) station in Colorado. Why might we be interested in studying litter and woody debris that falls to the ground in forests? Feel free to do a brief internet search if you feel you need more background information. (AI is allowed here, but put answers in your own words.)

Answer: Litter and woody debris represent the main way that carbon and nutrients move from plants into the soil. By studying what falls to the forest floor, we can learn a lot about nutrient cycling, soil carbon storage, and overall forest productivity. These inputs also influence soil organisms and long-term ecosystem processes, so tracking litter and woody debris helps us understand how forests function and how they respond to environmental change.

4. QUESTION: How is litter and woody debris sampled as part of the NEON network? Read the [NEON_Litterfall_UserGuide.pdf](#) document to learn more. List three pieces of salient information about the sampling methods here:

Answer: 1. NEON uses a standardized setup where litter traps are placed in fixed plots, and in many cases both elevated traps and ground-level sampling are used so different types of litter can be collected. 2. Litterfall and woody debris are clearly classified by size, and the traps have a known surface area, which makes it possible to compare biomass across different sites in a consistent way. 3. Samples are collected repeatedly throughout the year, with the timing adjusted to seasonal patterns like peak leaf fall, so changes over time and differences among sites can be compared reliably.

Obtain basic summaries of your data (Neonics)

5. What are the dimensions of the dataset?

```
# Dimensions = (rows, columns)
dim(Neonics)
```

```
## [1] 4623 30
```

6. Using the `summary` function on the “Effect” column, determine the most common effects that are studied. [Tip: The `sort()` command is useful for listing the values in order of magnitude...]

```
# Sort to see the most common effects first
effect_counts <- summary(Neonics$Effect)
effect_counts_sorted <- sort(effect_counts, decreasing = TRUE)

# Print the sorted counts
effect_counts_sorted
```

```
##      Population      Mortality      Behavior Feeding behavior
##      1803          1493          360          255
##      Reproduction      Development      Avoidance      Genetics
##      197             136             102             82
##      Enzyme(s)         Growth      Morphology      Immunological
##      62                38             22             16
```

```
##      Accumulation      Intoxication      Biochemistry      Cell(s)
##           12           12           11           9
##      Physiology      Histology      Hormone(s)
##           7           5           1
```

```
# Save the top 2 effect names for an auto-filled written answer
top2_effects <- names(effect_counts_sorted)[1:2]
top2_effects
```

```
## [1] "Population" "Mortality"
```

Question: Which two effects stand out as the most studied? Can you guess why these effects might specifically be of interest?

Answer: The two effects that appear most often are Population and Mortality. These effects are common because they're easy to measure and compare across studies. Many papers focus on basic questions like mortality or simple performance effects, so these outcomes show up more often than others.

- Using the `summary` function, determine the six most commonly studied species in the dataset (common name).[TIP: Explore the help on the `summary()` function, in particular the `maxsum` argument...]

```
# First attempt: use $ with backticks to access a column name that contains spaces.
top6_species <- summary(Neonics$`Species Common Name`, maxsum = 6)
top6_species
```

```
## Length Class Mode
##      0    NULL  NULL
```

```
# The first attempt returned NULL
# likely due to hidden characters or a non-exact match in the column name.
# To make the code more robust, explicitly search for the common-name column.
possible_common_cols <- c("Species Common Name", "Common.Name",
                          "CommonName", "Species.Common.Name")
common_col <- possible_common_cols[possible_common_cols %in% names(Neonics)][1]

if (is.na(common_col)) {
  stop("Could not find a common-name column. Check names(Neonics).")
}

# six most commonly studied species (common name)
top6_species <- summary(Neonics[[common_col]], maxsum = 6)
top6_species
```

```
##      Honey Bee      Parasitic Wasp Buff Tailed Bumblebee
##           667           285           183
##      Carniolan Honey Bee      Bumble Bee      (Other)
##           152           140           3196
```

Question: What do these species have in common? Why might they be of interest over other insects?

Answer: These species are commonly used because they're familiar to researchers and easier to study in controlled settings. Many of them also matter a lot ecologically or economically, like pollinators. As a result, studies on these insects are easier to compare and more useful for understanding impacts in real ecosystems.

8. The `Conc.1..Author` column, which lists the concentration of the neonicitoid dose, should include numeric values. What is the class of `Conc.1..Author.` column in the dataset, and why is it not numeric? [Tip: Viewing the dataframe may be helpful...]

```
# Check the data type (class) of the concentration column
class(Neonics$Conc.1..Author.)
```

```
## [1] "factor"
```

```
# Look at a few values to understand why it is not numeric
head(Neonics$Conc.1..Author., 50)
```

```
## [1] 27.2 19.7 47 25 13 268 170 28 48 40
## [11] 83 900 15.3 20.4 5 5 NR ~10 65.56 635.4
## [21] 239.1 NR/ 80.66 1075.7 75.27 636.5 183.4 1267.9 144.0/ 144.0/
## [31] 0.295 0.267 20.636 76.035 148.3 12/ 12/ 12/ 12/ 12/
## [41] 12/ 12/ 12/ 12/ 12/ 12.8 43.59 37.83 17.32 14.6
## 1006 Levels: <0.0004 <0.025 <0.088 <0.5 <1.5 <10/ <2.5/ <4.00 <5.00 ... NR/
```

Answer: The `Conc.1..Author.` column is not numeric; it is stored as a factor. This is because the concentration values are not always simple numbers. Many entries include ranges, inequality symbols like “<”, or text such as missing or not-reported values. Since these entries mix numbers with text, R cannot treat the column as purely numeric.

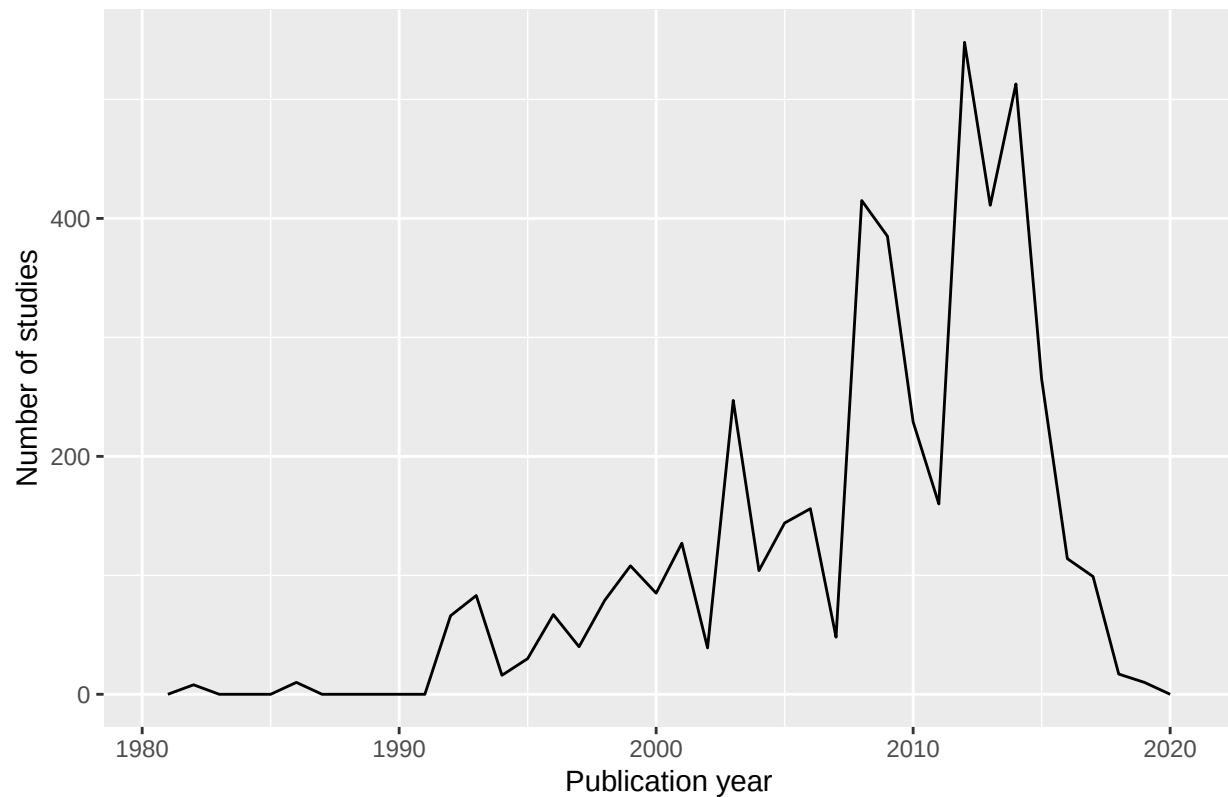
Explore your data graphically (Neonics)

9. Using `geom_freqpoly`, generate a plot of the number of studies conducted by publication year.

```
# If Publication.Year is a factor/character, convert it to numeric
Neonics <- Neonics %>%
  mutate(Publication.Year = as.numeric(as.character(Publication.Year)))

# Frequency polygon (like a smoothed histogram line)
ggplot(Neonics, aes(x = Publication.Year)) +
  geom_freqpoly(binwidth = 1) +
  labs(
    x = "Publication year",
    y = "Number of studies",
    title = "Number of neonicotinoid insect studies by year"
  )
```

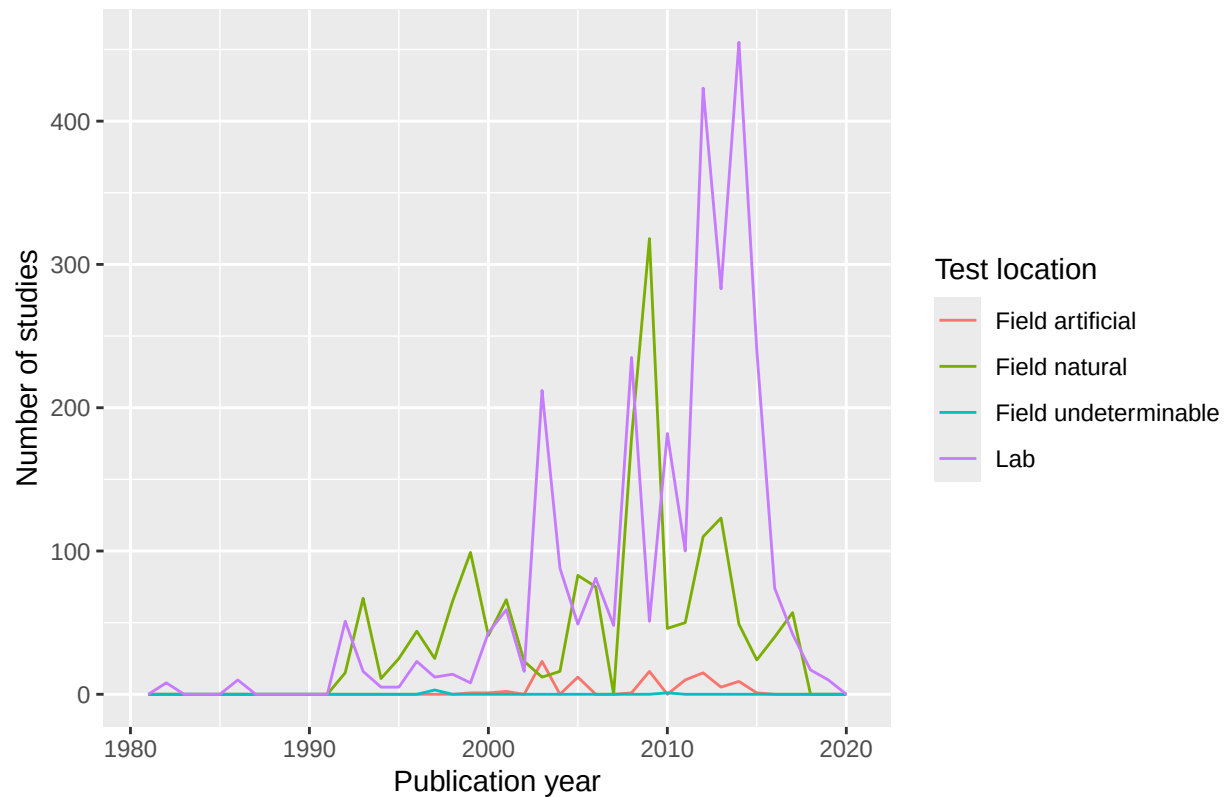
Number of neonicotinoid insect studies by year



10. Reproduce the same graph but now add a color aesthetic so that different Test.Location are displayed as different colors.

```
ggplot(Neonics, aes(x = Publication.Year, color = Test.Location)) +  
  geom_freqpoly(binwidth = 1) +  
  labs(  
    x = "Publication year",  
    y = "Number of studies",  
    color = "Test location",  
    title = "Studies by year, colored by test location"  
  )
```

Studies by year, colored by test location



```
# Count test locations (overall)
loc_counts <- sort(summary(Neonics$Test.Location), decreasing = TRUE)
head(loc_counts, 5)
```

```
##           Lab           Field natural           Field artificial
##           2860           1663           96
## Field undeterminable
##           4
```

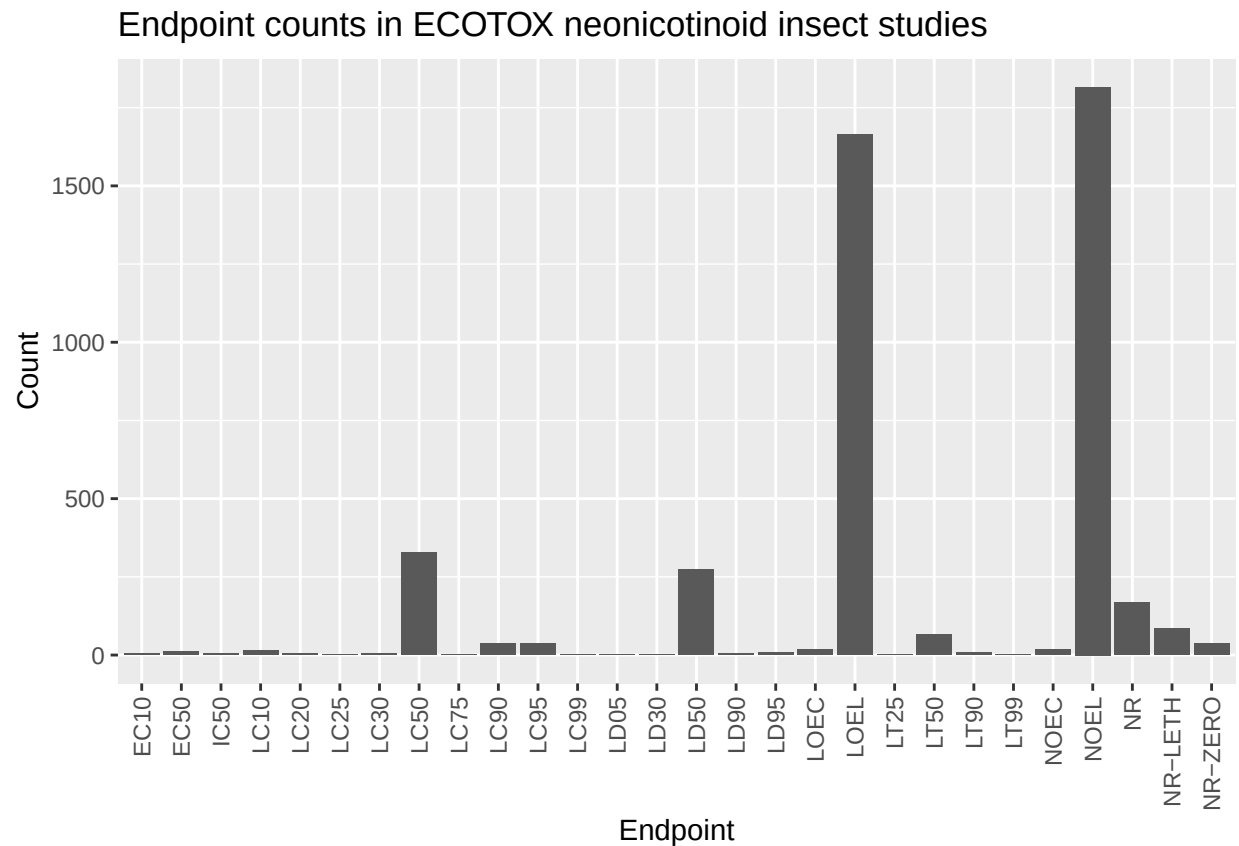
QUESTION: Interpret this graph. What are the most common test locations, and do they differ over time?

Answer: Overall, the most common test locations are Lab and Field natural. From the plot, it looks like the four locations stay common and rare across the early years, but the mix isn't perfectly stable. Certain locations become more visible and common in later years like Lab and Field natural, likely reflecting changes in where studies are conducted or which types of studies are more common in recent years.

11. Create a bar graph of Endpoint counts.

[TIP: Add `theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust=1))` to the end of your plot command to rotate and align the X-axis labels...]

```
ggplot(Neonics, aes(x = Endpoint)) +
  geom_bar() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1)) +
  labs(
    x = "Endpoint",
    y = "Count",
    title = "Endpoint counts in ECOTOX neonicotinoid insect studies"
  )
```



```
# Find the top two most common endpoints
endpoint_counts <- sort(summary(Neonics$Endpoint), decreasing = TRUE)
top2_endpoints <- names(endpoint_counts)[1:2]
top2_endpoints
```

```
## [1] "NOEL" "LOEL"
```

QUESTION: What are the two most common end points, and how are they defined? Consult the ECO-TOX_CodeAppendix (p.721) for more information.

Answer: The two most common endpoints are NOEL and LOEL. NOEL (No-Observed-Effect Level) is the highest dose or concentration where no statistically significant effects are observed compared to the control group. LOEL (Lowest-Observed-Effect Level) is the lowest dose or concentration at which a statistically significant effect is first observed compared to the control.

Explore your data (Litter)

12. Determine the class of `collectDate`. Is it a date? If not, change to a date and confirm the new class of the variable. Using the `unique` function, determine which dates litter was sampled in August 2018.

```
# Check the current class of collectDate
```

```
class(Litter$collectDate)
```

```
## [1] "factor"
```

```
# It is factor, then convert it
```

```
Litter$collectDate <- as.Date(Litter$collectDate)
```

```
# Confirm the new class
```

```
class(Litter$collectDate)
```

```
## [1] "Date"
```

```
# List the dates in August 2018 when samples were collected
```

```
aug_dates <- unique(Litter$collectDate[  
  Litter$collectDate >= as.Date("2018-08-01") &  
  Litter$collectDate <= as.Date("2018-08-31")  
)
```

```
aug_dates
```

```
## [1] "2018-08-02" "2018-08-30"
```

13. Using the `unique` function, list the different `plotIDs` sampled at Niwot Ridge.

```
# List all distinct plotIDs at this site/data extract
```

```
unique(Litter$plotID)
```

```
## [1] NIWO_061 NIWO_064 NIWO_067 NIWO_040 NIWO_041 NIWO_063 NIWO_047 NIWO_051
```

```
## [9] NIWO_058 NIWO_046 NIWO_062 NIWO_057
```

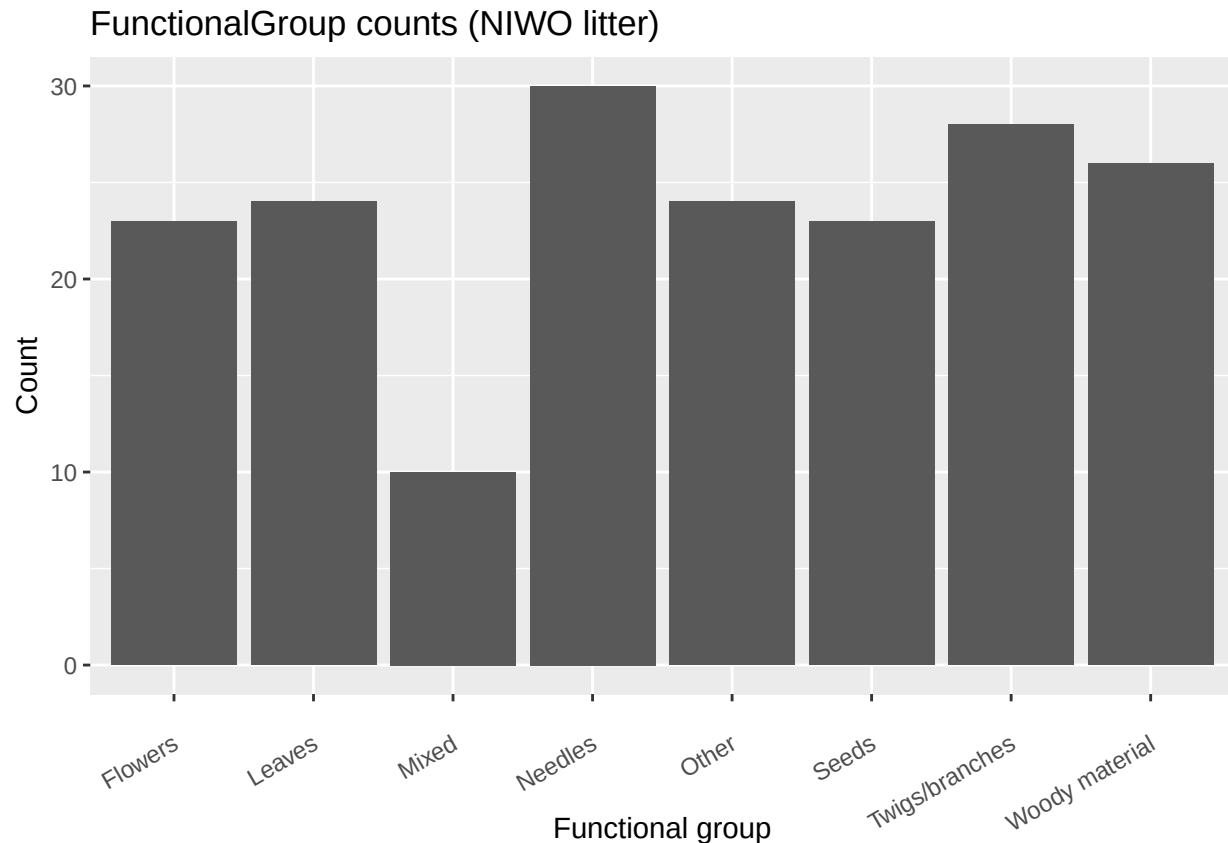
```
## 12 Levels: NIWO_040 NIWO_041 NIWO_046 NIWO_047 NIWO_051 NIWO_057 ... NIWO_067
```

QUESTION: How is the information obtained from `unique` different from that obtained from `summary`?

Answer: `unique()` just lists the different values that show up (like a “set” of plot IDs). `summary()` is more of a quick overview cause it can give counts. So `unique()` tells me what exists, while `summary()` tells me how it’s distributed.

14. Create a bar graph of `functionalGroup` counts. This shows you what type of litter is collected at the Niwot Ridge sites. Notice that litter types are fairly equally distributed across the Niwot Ridge sites.

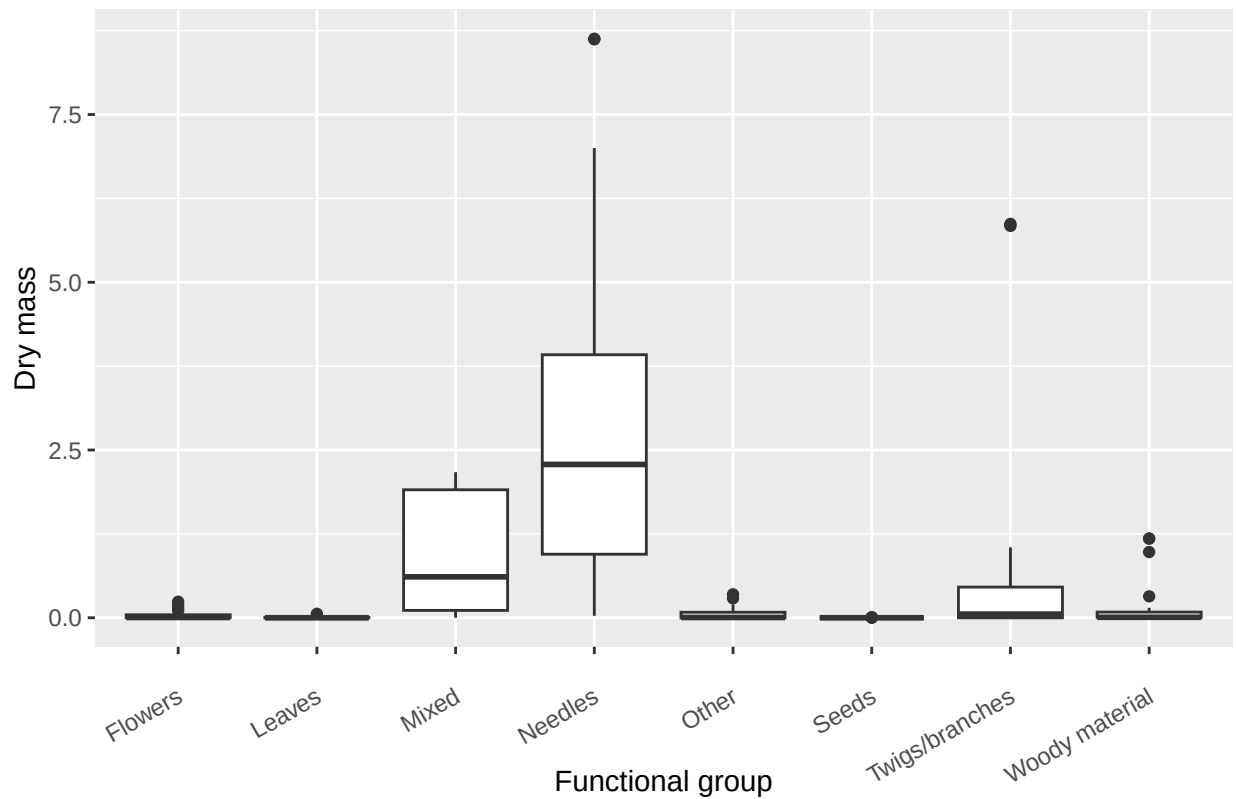
```
# Count how many samples are in each functionalGroup
ggplot(Litter, aes(x = functionalGroup)) +
  geom_bar() +
  labs(
    x = "Functional group",
    y = "Count",
    title = "FunctionalGroup counts (NIWO litter)"
  ) +
  theme(axis.text.x = element_text(angle = 30, vjust = 0.7, hjust = 1))
```



15. Using `geom_boxplot` and `geom_violin`, create a boxplot and a violin plot of `dryMass` by `functionalGroup`.

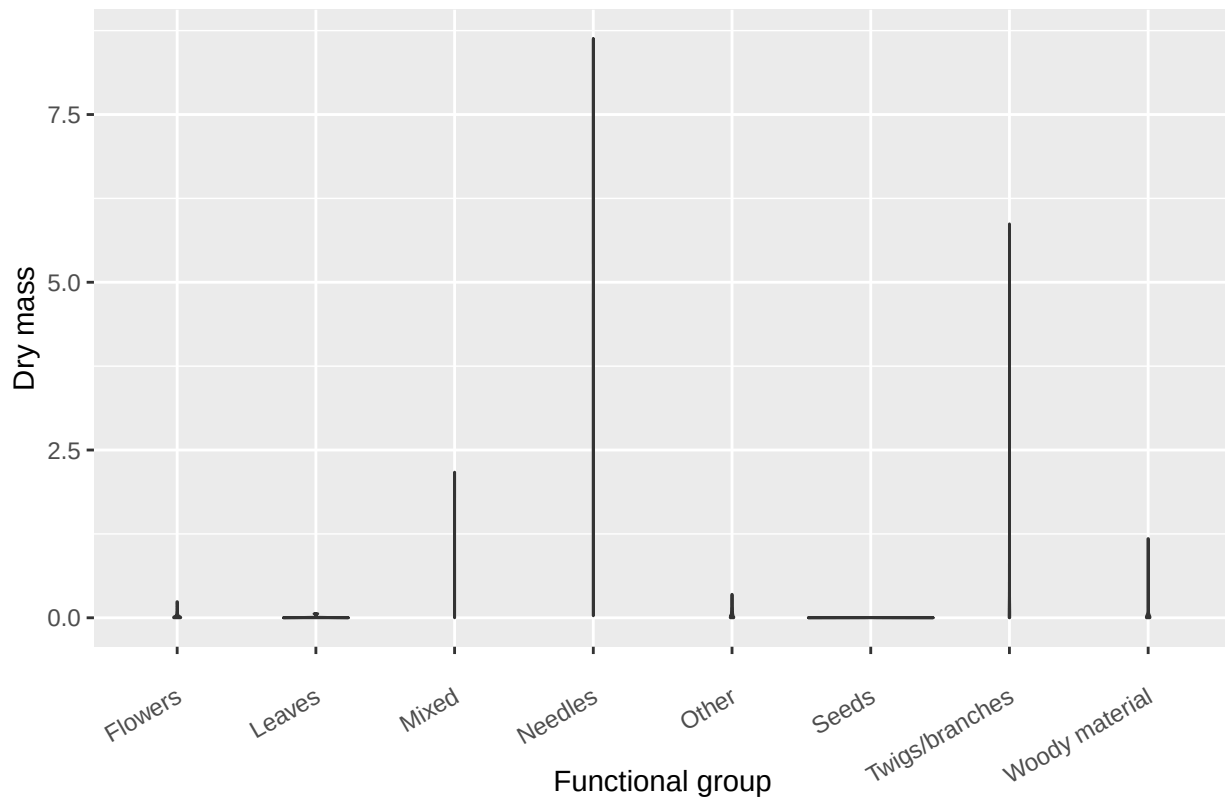
```
# Boxplot
ggplot(Litter, aes(x = functionalGroup, y = dryMass)) +
  geom_boxplot() +
  labs(
    x = "Functional group",
    y = "Dry mass",
    title = "Dry mass by functional group (boxplot)"
  ) +
  theme(axis.text.x = element_text(angle = 30, vjust = 0.7, hjust = 1))
```

Dry mass by functional group (boxplot)



```
# Violin plot
ggplot(Litter, aes(x = functionalGroup, y = dryMass)) +
  geom_violin() +
  labs(
    x = "Functional group",
    y = "Dry mass",
    title = "Dry mass by functional group (violin)"
  ) +
  theme(axis.text.x = element_text(angle = 30, vjust = 0.7, hjust = 1))
```

Dry mass by functional group (violin)



```
# Quick summary to see which groups tend to have higher dryMass
biomass_summary <- Litter %>%
  group_by(functionalGroup) %>%
  summarize(
    median_dryMass = median(dryMass, na.rm = TRUE),
    mean_dryMass = mean(dryMass, na.rm = TRUE),
    n = n()
  ) %>%
  arrange(desc(median_dryMass))

biomass_summary
```

```
## # A tibble: 8 x 4
##   functionalGroup median_dryMass mean_dryMass    n
##   <fct>             <dbl>         <dbl> <int>
## 1 Needles           2.28           2.70     30
## 2 Mixed             0.61           0.965    10
## 3 Twigs/branches    0.055          0.623    28
## 4 Woody material    0.0075         0.165    26
## 5 Flowers           0.005          0.0398   23
## 6 Other             0.005          0.0581   24
## 7 Leaves            0           0.00375   24
## 8 Seeds             0           0.00109   23
```

QUESTION: Why is the boxplot a more effective visualization option than the violin plot in this case?

Answer: The boxplot makes it really easy to compare groups because we can immediately see the median, spread, and outliers. The violin plot here it doesn't add much so it's harder to quickly tell which group is higher, especially if sample sizes are uneven or the distribution shapes are noisy.

QUESTION: What type(s) of litter tend to have the highest biomass at these sites?

Answer: Needles clearly have the highest biomass at these sites. In the boxplot, the needles group has the highest median dry mass and the widest spread, with several large values compared to the other groups. Mixed litter and twigs/branches also tend to have higher biomass than flowers or leaves, but they are still much lower than needles overall.